Date.prototype.setHours()

📄 Título

Date.prototype.setHours() – El establecedor de las horas

📝 Definición

`Date.prototype.setHours()` es un método de JavaScript que establece las **horas** (0-23) de un objeto `Date` según la **hora local**, pudiendo también establecer opcionalmente los minutos, segundos y milisegundos.

🧠 ¿Qué es?

Es el método estándar para **modificar la hora** de una fecha en JavaScript.

Permite cambiar la hora manteniendo el resto de componentes (año, mes, día), con la capacidad de manejar **overflows** automáticamente.

El método es **mutable**: modifica el objeto `Date` original en lugar de crear uno nuevo.

🎯 ¿Para qué sirve?

- Cambiar la **hora** de una fecha existente.
- Sumar o restar **horas** a una fecha.
- Implementar **temporizadores** y **cronómetros**.
- Ajustar horas para **conversiones de zona horaria**.
- Establecer **horas exactas** en fechas (ej: medianoche).
- Manejar **ajustes automáticos** de días (overflows).
- Implementar cálculos de **intervalos de tiempo** en horas.
- Crear **marcas de tiempo** con precisión horaria.

⚙️ ¿Cómo funciona?

El método establece las horas (0-23), y opcionalmente los minutos, segundos y milisegundos, basado en la **hora local**.

Comportamiento:

- Las horas deben ser un número entre 0 y 23.
- Si solo se pasan horas, se mantienen los minutos, segundos y milisegundos existentes.
- Los valores fuera de rango se **ajustan automáticamente**:
  - Horas > 23 → se ajusta al día siguiente.
  - Horas < 0 → se ajusta al día anterior.
  - Ej: `setHours(25)` → 1:00 AM del día siguiente.
  - Ej: `setHours(-1)` → 11:00 PM del día anterior.
- Los minutos, segundos y milisegundos también se ajustan automáticamente si se pasan fuera de rango.

Retorna: El **timestamp** en milisegundos del objeto modificado.

📝 Sintaxis / Estructura (JavaScript)

```
date.setHours(hours)
date.setHours(hours, minutes)
date.setHours(hours, minutes, seconds)
date.setHours(hours, minutes, seconds, milliseconds)
```

Parámetros:

- `hours` (obligatorio): Número entero para las horas (0-23, se aceptan valores fuera de rango con ajuste automático).
- `minutes` (opcional): Número entero para los minutos (0-59).
- `seconds` (opcional): Número entero para los segundos (0-59).
- `milliseconds` (opcional): Número entero para los milisegundos (0-999).

Retorna: El **timestamp** en milisegundos (del objeto modificado).

💡 Ejemplos

```
// 1. Uso básico - establecer solo las horas
const date = new Date(2024, 11, 25, 10, 30);
console.log(date.getHours()); // 10

date.setHours(15);
console.log(date.getHours()); // 15
console.log(date.getMinutes()); // 30 (mantiene los minuto
s)

// 2. Establecer horas, minutos, segundos y milisegundos
const date2 = new Date(2024, 11, 25, 10, 30, 45, 500);
date2.setHours(15, 20, 30, 200);
console.log(date2.getHours()); // 15
console.log(date2.getMinutes()); // 20
console.log(date2.getSeconds()); // 30
console.log(date2.getMilliseconds()); // 200

// 3. Ajuste automático - horas fuera de rango
const date3 = new Date(2024, 11, 25, 10, 30);
date3.setHours(25);
console.log(date3.getHours()); // 1
console.log(date3.getDate()); // 26 (día siguiente)

// 4. Ajuste automático - horas negativas
const date4 = new Date(2024, 11, 25, 10, 30);
date4.setHours(-1);
console.log(date4.getHours()); // 23
console.log(date4.getDate()); // 24 (día anterior)

// 5. Sumar horas a una fecha
function addHours(date, hours) {
  const newDate = new Date(date);
  newDate.setHours(newDate.getHours() + hours);
  return newDate;
}
```

```
const start = new Date(2024, 11, 25, 10, 30);
console.log(addHours(start, 5)); // 15:30
console.log(addHours(start, 25)); // 11:30 (día siguiente)
console.log(addHours(start, -3)); // 07:30

// 6. Establecer medianoche (00:00:00.000)
function setMidnight(date) {
  const newDate = new Date(date);
  newDate.setHours(0, 0, 0, 0);
  return newDate;
}

const d = new Date(2024, 11, 25, 10, 30, 45, 500);
console.log(setMidnight(d)); // 2024-12-25 00:00:00.000

// 7. Establecer mediodía (12:00:00.000)
function setNoon(date) {
  const newDate = new Date(date);
  newDate.setHours(12, 0, 0, 0);
  return newDate;
}

console.log(setNoon(new Date(2024, 11, 25, 10, 30, 45, 50
0))); // 12:00:00.000

// 8. Fin del día (23:59:59.999)
function setEndOfDay(date) {
  const newDate = new Date(date);
  newDate.setHours(23, 59, 59, 999);
  return newDate;
}

console.log(setEndOfDay(new Date(2024, 11, 25, 10, 30)));
// 23:59:59.999

// 9. Verificar si una fecha es exactamente medianoche
function isMidnight(date) {
  return date.getHours() === 0 &&
```

```
          date.getMinutes() === 0 &&
          date.getSeconds() === 0 &&
          date.getMilliseconds() === 0;
}

console.log(isMidnight(new Date(2024, 11, 25, 0, 0, 0,
0))); // true
console.log(isMidnight(new Date(2024, 11, 25, 10, 30))); //
false

// 10. Ajuste automático con minutos, segundos y ms
const date5 = new Date(2024, 11, 25, 10, 30, 45, 500);
date5.setHours(15, 70, 80, 1500);
console.log(date5.getHours()); // 16 (15 + 1 por minutos)
console.log(date5.getMinutes()); // 11 (70 - 60 = 10 + 1 po
r segundos)
console.log(date5.getSeconds()); // 21 (80 - 60 = 20 + 1 po
r ms)
console.log(date5.getMilliseconds()); // 500 (1500 - 1000 =
500)

// 11. Mantener minutos, segundos y ms existentes
const date6 = new Date(2024, 11, 25, 10, 30, 45, 500);
date6.setHours(15);
console.log(date6.getHours()); // 15
console.log(date6.getMinutes()); // 30 (mantiene)
console.log(date6.getSeconds()); // 45 (mantiene)
console.log(date6.getMilliseconds()); // 500 (mantiene)

// 12. Comparar solo horas
function sameHour(date1, date2) {
  return date1.getHours() === date2.getHours();
}

console.log(sameHour(
  new Date(2024, 11, 25, 10, 30),
  new Date(2024, 11, 25, 10, 45)
)); // true
```

```
// 13. Generar fechas por hora
function generateDatesByHour(year, month, day, minutes = 0)
{
  const dates = [];
  for (let hour = 0; hour < 24; hour++) {
    const date = new Date(year, month, day, hour, minutes);
    dates.push(date);
  }
  return dates;
}

console.log(generateDatesByHour(2024, 11, 25, 30)); // 24 f
echas cada hora con :30

// 14. Calcular diferencia en horas
function hoursBetween(date1, date2) {
  const diff = Math.abs(date2.getTime() - date1.getTime());
  return Math.floor(diff / (1000 * 60 * 60));
}

console.log(hoursBetween(
  new Date(2024, 11, 25, 10, 0),
  new Date(2024, 11, 25, 15, 0)
)); // 5

// 15. Redondear a la hora más cercana
function roundToNearestHour(date) {
  const newDate = new Date(date);
  const minutes = newDate.getMinutes();
  if (minutes >= 30) {
    newDate.setHours(newDate.getHours() + 1, 0, 0, 0);
  } else {
    newDate.setHours(newDate.getHours(), 0, 0, 0);
  }
  return newDate;
}
```

```
console.log(roundToNearestHour(new Date(2024, 11, 25, 10, 29))); // 10:00
console.log(roundToNearestHour(new Date(2024, 11, 25, 10, 30))); // 11:00
```

⚠️ Errores comunes

- **Pasar horas fuera de rango sin entender el ajuste**:

```
const date = new Date(2024, 11, 25, 10, 30);
date.setHours(25);
console.log(date.getHours()); // 1
console.log(date.getDate()); // 26 (¡cambió el día!)
```

- **Confundir el orden de parámetros**:

```
// setHours(hours, minutes, seconds, milliseconds)
// ❌ NO setHours(minutes, hours, seconds, milliseconds)
```

- **No considerar la zona horaria**:

```
// setHours() trabaja en hora local
const date = new Date('2024-12-25T10:30:00Z');
date.setHours(15);
// Modifica la hora local, no la UTC
```

- **Olvidar que setHours() modifica el objeto original**:

```
const original = new Date(2024, 11, 25, 10, 30);
const copy = original;
copy.setHours(15);
console.log(original.getHours()); // 15 (¡también modificado!)
```

```
// Para evitar: const copy = new Date(original);
```

💼 Casos de uso

1. **Sistema de turnos y horarios laborales**:

```
class ShiftManager {
  constructor() {
    this.shifts = [];
  }

  createShift(name, startHour, endHour, days = [1,2,3,4,
5]) {
    this.shifts.push({
      name,
      startHour,
      endHour,
      days,
      active: true
    });

    return this.shifts[this.shifts.length - 1];
  }

  isOnShift(date) {
    const hour = date.getHours();
    const day = date.getDay(); // 0 = domingo

    return this.shifts
      .filter(s => s.active)
      .some(s => {
        const isDayMatch = s.days.includes(day);
        const isHourInRange = hour >= s.startHour && hou
r < s.endHour;
        return isDayMatch && isHourInRange;
      });
```

```
  }

  getCurrentShift(date = new Date()) {
    const hour = date.getHours();
    const day = date.getDay();

    return this.shifts
      .filter(s => s.active)
      .find(s => {
        const isDayMatch = s.days.includes(day);
        const isHourInRange = hour >= s.startHour && hou
r < s.endHour;
        return isDayMatch && isHourInRange;
      });
  }

  getScheduleForDay(date) {
    const day = date.getDay();
    return this.shifts
      .filter(s => s.active && s.days.includes(day))
      .sort((a, b) => a.startHour - b.startHour);
  }
}

const shifts = new ShiftManager();
shifts.createShift('Mañana', 9, 14, [1,2,3,4,5]);
shifts.createShift('Tarde', 14, 19, [1,2,3,4,5]);
shifts.createShift('Fin de semana', 10, 18, [0,6]);
console.log(shifts.getScheduleForDay(new Date(2024, 6, 1
5)));
```

2. **Sistema de disponibilidad horaria**:

```
class AvailabilityManager {
  constructor() {
    this.schedule = {};
  }
```

```
setAvailability(day, startHour, endHour) {
  if (!this.schedule[day]) {
    this.schedule[day] = [];
  }

  this.schedule[day].push({
    startHour,
    endHour
  });

  return this.schedule[day];
}

isAvailable(date) {
  const day = date.getDay();
  const hour = date.getHours();

  if (!this.schedule[day]) return false;

  return this.schedule[day].some(slot =>
    hour >= slot.startHour && hour < slot.endHour
  );
}

getAvailableSlots(date, duration = 1) {
  const day = date.getDay();
  const slots = [];

  if (!this.schedule[day]) return slots;

  const hour = date.getHours();
  const endHour = 23;

  for (let h = hour; h < endHour; h += duration) {
    const slotDate = new Date(date);
    slotDate.setHours(h, 0, 0, 0);
```

```
      if (this.isAvailable(slotDate)) {
        slots.push({
          start: new Date(slotDate),
          end: new Date(slotDate.getTime() + duration *
60 * 60 * 1000),
          time: `${h.toString().padStart(2, '0')}:00
-${(h + duration).toString().padStart(2, '0')}:00`
        });
      }
    }

    return slots;
  }

  getWeeklyAvailability() {
    const days = ['Domingo', 'Lunes', 'Martes', 'Miércol
es', 'Jueves', 'Viernes', 'Sábado'];
    const result = {};

    for (let day = 0; day < 7; day++) {
      result[days[day]] = this.schedule[day] || [];
    }

    return result;
  }
}

const availability = new AvailabilityManager();
availability.setAvailability(1, 9, 18); // Lunes 9-18
availability.setAvailability(2, 9, 18); // Martes 9-18
availability.setAvailability(3, 9, 18); // Miércoles 9-1
8
availability.setAvailability(4, 9, 18); // Jueves 9-18
availability.setAvailability(5, 9, 15); // Viernes 9-15
console.log(availability.getAvailableSlots(new Date(202
4, 6, 15)));
```

3. **Sistema de logs con marca de tiempo**:

```
class ActivityLogger {
  constructor() {
    this.activities = [];
  }

  logActivity(user, action, details = null) {
    const now = new Date();
    const timestamp = new Date(now);
    timestamp.setMilliseconds(0);

    const hour = timestamp.getHours();
    const minutes = timestamp.getMinutes();

    this.activities.push({
      user,
      action,
      details,
      timestamp,
      hour,
      minutes,
      time: timestamp.toLocaleTimeString('es-ES', {
        hour: '2-digit',
        minute: '2-digit',
        second: '2-digit'
      })
    });

    return this.activities[this.activities.length - 1];
  }

  getActivitiesByHour(hour) {
    return this.activities.filter(a => a.hour === hour);
  }

  getActivitiesByTimeRange(startHour, endHour) {
    return this.activities.filter(a =>
```

```
      a.hour >= startHour && a.hour <= endHour
    );
  }

  getActivityStats() {
    const stats = {};
    for (let hour = 0; hour < 24; hour++) {
      stats[hour] = {
        count: this.activities.filter(a => a.hour === ho
ur).length,
        users: [...new Set(this.activities.filter(a =>
a.hour === hour).map(a => a.user))]
      };
    }
    return stats;
  }

  getPeakHours() {
    const stats = this.getActivityStats();
    const max = Math.max(...Object.values(stats).map(s =
> s.count));
    return Object.entries(stats)
      .filter(([_, value]) => value.count === max)
      .map(([hour]) => parseInt(hour));
  }
}

const logger = new ActivityLogger();
logger.logActivity('user1', 'login');
logger.logActivity('user2', 'page_view');
logger.logActivity('user1', 'logout');
console.log(logger.getActivityStats());
console.log(logger.getPeakHours());
```

4. **Sistema de programación de tareas**:

```
class TaskScheduler {
  constructor() {
    this.tasks = [];
    this.interval = null;
  }

  scheduleTask(name, hour, minute = 0, frequency = 'dail
y') {
    const now = new Date();
    const nextRun = new Date(now);
    nextRun.setHours(hour, minute, 0, 0);

    // Si ya pasó la hora, programar para mañana
    if (nextRun < now) {
      nextRun.setDate(nextRun.getDate() + 1);
    }

    this.tasks.push({
      name,
      hour,
      minute,
      frequency,
      nextRun,
      lastRun: null,
      active: true
    });

    this.startScheduler();
    return this.tasks[this.tasks.length - 1];
  }

  startScheduler() {
    if (this.interval) return;

    this.interval = setInterval(() => {
      const now = new Date();
      this.tasks.forEach(task => {
        if (!task.active) return;
```

```
        const taskTime = new Date(now);
        taskTime.setHours(task.hour, task.minute, 0, 0);

        // Verificar si es hora de ejecutar
        if (taskTime <= now && (!task.lastRun || taskTim
e > task.lastRun)) {
          this.executeTask(task);
        }
      });
    }, 60000);
  }

  executeTask(task) {
    task.lastRun = new Date();
    console.log(`✅${task.name} ejecutada a las${task.ho
ur}:${task.minute}`);

    // Actualizar próxima ejecución
    const next = new Date(task.lastRun);
    switch(task.frequency) {
      case 'daily':
        next.setDate(next.getDate() + 1);
        break;
      case 'weekly':
        next.setDate(next.getDate() + 7);
        break;
      case 'monthly':
        next.setMonth(next.getMonth() + 1);
        break;
    }
    task.nextRun = next;
  }

  getTasksByHour(hour) {
    return this.tasks.filter(t => t.hour === hour);
  }
```

```
  getUpcomingTasks() {
    return this.tasks
      .filter(t => t.active)
      .sort((a, b) => a.nextRun - b.nextRun)
      .slice(0, 10);
  }

  stopScheduler() {
    if (this.interval) {
      clearInterval(this.interval);
      this.interval = null;
    }
  }
}

const scheduler = new TaskScheduler();
scheduler.scheduleTask('Backup DB', 2, 0);
scheduler.scheduleTask('Reporte diario', 9, 30);
scheduler.scheduleTask('Limpiar logs', 23, 59);
console.log(scheduler.getUpcomingTasks());
```

5. **Sistema de métricas horarias**:

```
class HourlyMetrics {
  constructor() {
    this.metrics = {};
    this.historical = [];
  }

  recordMetric(name, value) {
    const now = new Date();
    const hour = now.getHours();
    const timestamp = new Date(now);
    timestamp.setMinutes(0, 0, 0);

    if (!this.metrics[hour]) {
      this.metrics[hour] = {};
```

```
    }

    if (!this.metrics[hour][name]) {
      this.metrics[hour][name] = {
        values: [],
        count: 0,
        sum: 0
      };
    }

    this.metrics[hour][name].values.push(value);
    this.metrics[hour][name].count++;
    this.metrics[hour][name].sum += value;

    this.historical.push({
      name,
      value,
      hour,
      timestamp
    });
  }

  getHourlyStats(hour) {
    if (!this.metrics[hour]) return null;

    const stats = {};
    for (const [name, data] of Object.entries(this.metri
cs[hour])) {
      stats[name] = {
        count: data.count,
        average: data.sum / data.count,
        total: data.sum,
        min: Math.min(...data.values),
        max: Math.max(...data.values)
      };
    }
    return stats;
  }
```

```
  getPeakHour(name) {
    let maxCount = 0;
    let peakHour = 0;

    for (let hour = 0; hour < 24; hour++) {
      if (this.metrics[hour] && this.metrics[hour][nam
e]) {
        const count = this.metrics[hour][name].count;
        if (count > maxCount) {
          maxCount = count;
          peakHour = hour;
        }
      }
    }

    return peakHour;
  }

  getHourlyComparison(name1, name2) {
    const comparison = {};

    for (let hour = 0; hour < 24; hour++) {
      const metric1 = this.metrics[hour]?.[name1];
      const metric2 = this.metrics[hour]?.[name2];

      comparison[hour] = {
        [name1]: metric1 ? metric1.sum : 0,
        [name2]: metric2 ? metric2.sum : 0,
        difference: (metric1?.sum || 0) - (metric2?.sum
|| 0)
      };
    }

    return comparison;
  }

  reset() {
```

```
    this.metrics = {};
    this.historical = [];
  }
}

const metrics = new HourlyMetrics();
metrics.recordMetric('requests', 100);
metrics.recordMetric('requests', 150);
metrics.recordMetric('errors', 5);
metrics.recordMetric('requests', 200);
console.log(metrics.getHourlyStats(new Date().getHours
()));
```

🧩 Relación con otros conceptos

- `Date.prototype.getHours()`: Obtiene las horas.
- `Date.prototype.setMinutes()`: Establece los minutos.
- `Date.prototype.setSeconds()`: Establece los segundos.
- `Date.prototype.setMilliseconds()`: Establece los milisegundos.
- `Date.prototype.getTime()`: Obtiene el timestamp.
- `Date.prototype.setUTCHours()`: Versión en UTC.

🧠 Analogía

Imagina que `setHours()` es como un **reloj analógico ajustable**. Puedes mover la manecilla de las horas para cambiar la hora, y si das una vuelta completa (24 horas), el día cambia automáticamente. Es una forma simple de ajustar la hora manteniendo todo sincronizado.

📚 Fuente

- MDN Web Docs – Date.prototype.setHours()
- ECMAScript Specification – Date.prototype.setHours